
BudgetFlow: Value-Aware Budget Governance for Agent Tasks

Fengde Ding¹

fengde_ding@uic.edu.cn

¹University of International Relations

Abstract

Multi-step AI agent tasks consume variable amounts of model budget, and tasks in a batch often have unequal value. We study *Batch-Level Task Budget Allocation*: a task batch shares one hard budget, and scarce model opportunities should flow toward the tasks that create the most verified value.

We introduce BudgetFlow, a value-aware budget governance framework for task batches. Its general interface combines task-level signals—Task Value, Estimated Token Cost, Model Fit, and a Verifier—with batch-level Shared Budget state. BudgetFlow uses the first three task signals and the Shared Budget to allocate budget and model tiers, while the Verifier settles outcomes. This separates allocation signals from outcome settlement, so the same interface can describe coding, support, document, spreadsheet, and research task batches once each domain supplies a trusted verifier.

We evaluate BudgetFlow on SWE-bench-style verified bug-fixing tasks under fixed task sets and shared hard budgets. The paper reports Resolved Rate, Total Resolved Value (TRV), and TRV per Dollar. Across one 4-policy run and three 5-policy runs over 30-task batches, BudgetFlow produces two value-aware frontier results: with metadata-only estimates, it resolves higher value than the strong-model-only baseline at about 16% lower spend; with prior-batch observations, it leads all three reported metrics. The other main batches identify Strong Coverage cases, where broad strong-model use is already cost-effective under the cap. To make this evidence interpretable, we propose a frontier-mode framework with three modes: Strong Coverage, Model Fit, and Value-Aware Budget Flow. The third mode is the central target of BudgetFlow.

1. Introduction

AI agents increasingly execute multi-step tasks: they inspect context, call tools, edit artifacts, run checks, and decide whether another model call is worth the cost. These tasks consume budget unevenly. One task may finish after a short reasoning loop; another may require repeated search, repair, and verification. In a batch, this variation means that a fixed

model budget can be exhausted before every task receives the same opportunity to succeed.

A central issue is that tasks rarely have equal value. A support ticket with an imminent service-level agreement, a security patch, a revenue-critical spreadsheet fix, and a routine rewrite should receive different budget treatment when they are submitted in the same batch. Once model calls become a scarce shared resource, the system needs an allocation policy in addition to a model router.

This paper studies *Batch-Level Task Budget Allocation*. The general setting has many tasks, one shared hard budget, model tiers with different costs and capabilities, and an outcome rule that decides whether a task is resolved. The objective is to allocate scarce model opportunities so the batch produces more verified task value under the same resource constraint.

The operational decision is simple: which task should receive the next scarce model opportunity, under the same Shared Budget?

This setting admits several frontier modes. Strong Coverage describes batches where broad use of the strong model is cost-effective. Model Fit describes batches where task-to-model matching explains most of the useful routing signal. BudgetFlow targets Value-Aware Budget Flow: when tasks differ in value, estimated token cost, and model-tier upside, the batch policy should allocate scarce model opportunities across the full task batch.

BudgetFlow is a budget governance framework for this setting. It uses four task-level signals and one batch-level budget state:

- **Task Value:** the pre-registered value gained if the task is resolved;
- **Estimated Token Cost:** the pre-run estimate of token cost used for cap planning;
- **Model Fit:** evidence about how well each model tier fits the task;
- **Verifier:** the outcome rule that maps a completed attempt to resolved or unresolved;

- **Shared Budget:** the hard batch-level resource constraint shared by all tasks.

The first three task-level signals and the Shared Budget drive allocation decisions. The Verifier settles outcomes and defines the objective. This separation is the core interface. The policy can change how it routes, caps, assigns model opportunities, or stops tasks, while the outcome contract remains explicit. Task-source labels such as coding, marketing, support, or document production can be recorded as context; strong-model access is governed by declared value, estimated token cost, model-fit evidence, and the shared budget.

We instantiate this formulation on SWE-bench-style verified coding tasks. SWE-bench is useful here because it gives a strict task-batch testbed: each task is a real bug-fixing issue, each attempt has measurable model spend, and each outcome is settled by a test-based harness. The benchmark is the evidence domain, while the allocation interface is the research object.

The paper uses three primary metrics. **Resolved Rate** anchors the evaluation to standard verified success. **Total Resolved Value** (TRV) sums pre-registered Task Value over resolved tasks. **TRV per Dollar** measures value efficiency under the shared hard budget.

The central empirical result is the BudgetFlow frontier-mode evidence. In a 4-policy metadata-only batch, BudgetFlow reaches TRV 18.50 at \$7.98, while strong-model-only reaches TRV 18.00 at \$9.46 and resolves one additional task. In a 5-policy batch with prior-batch observations, BudgetFlow leads Resolved Rate, TRV, and TRV per Dollar. The remaining main batches show Strong Coverage: broad use of the strong model forms the observed frontier, once with a larger lead and once with a close lead. Serving-cost sensitivity shows the Model Fit case: under extreme KV-cache discounts, learned routing becomes the best TRV-per-Dollar point while BudgetFlow keeps higher TRV. Together, these outcomes show how the frontier is shaped by Shared Budget pressure, Task Value, Estimated Token Cost, and Model Fit.

This paper makes three contributions:

1. We formulate Batch-Level Task Budget Allocation as a value-governance problem with pre-registered Task Value, Estimated Token Cost, Model Fit, a shared hard budget, and a Verifier-settled objective.
2. We present a generalizable BudgetFlow framework and interface that allocate model opportunities through Task Value, Estimated Token Cost, Model Fit, Shared Budget, and a Verifier-settled outcome contract across task domains.
3. We propose a frontier-mode framework for interpret-

ing shared-budget agent batches, identifying **Strong Coverage, Model Fit, and Value-Aware Budget Flow** as three frontier modes. **Value-Aware Budget Flow** is the central mode targeted by BudgetFlow.

Figure 1 summarizes the BudgetFlow loop. Adjacent systems decide which model answers one request or how one task spends its own budget. BudgetFlow instead repeatedly allocates scarce budget and strong-model opportunities across a task pool under one shared hard budget, then settles outcomes with a Verifier and updates remaining budget and verified TRV.

2. Related Work

We organize related work by the decision problem it solves. Conference venues are not a ranking signal: newer papers are placed only by what decision they make.

Per-request cost-quality routing. RouteLLM learns per-query routers from preference data (Ong et al., 2025). Cascade Routing studies model routing and cascading for individual queries (Dekoninck et al., 2025). RouteNLP targets enterprise query routing under quality constraints (Guo et al., 2026). UCCI calibrates uncertainty for cost-optimal cascade routing (Kotte, 2026). ZeroRouter maps requests and models into a shared latent space for multi-objective routing and low-cost onboarding of new models (Yan et al., 2026). Capability Instruction Tuning / MODEL-SAT selects models from capability instructions (Zhang et al., 2025). ICL-Router and CP-Router further study request-level routing with in-context model representations or uncertainty-aware LLM/LRM choice (Wang et al., 2026; Su et al., 2026). These methods make model selection a first-class cost-quality problem for one prompt or one request. They do not allocate one shared hard budget across many tasks with unequal value.

Within-task budgeted agent control. INTENT plans costly tool use inside a single agent task under that task’s own budget (Liu et al., 2026). BAMAS builds a multi-agent system for one task under a per-task cost cap by choosing models and collaboration structure (Yang et al., 2026). On-line Multi-LLM Selection adapts model choice over multi-turn interaction with a per-dialogue budget when the next prompt evolves as a black box (Poon et al., 2026). CCPO orchestrates cheaper and stronger models inside one question under a reliability constraint (Si et al., 2026). STEER switches between small and large models at reasoning steps using confidence signals (Lee et al., 2026). These methods optimize spend or model switches inside one task or one interaction. Even when a budget exists, it belongs to that task or dialogue, not to a batch of competing tasks.

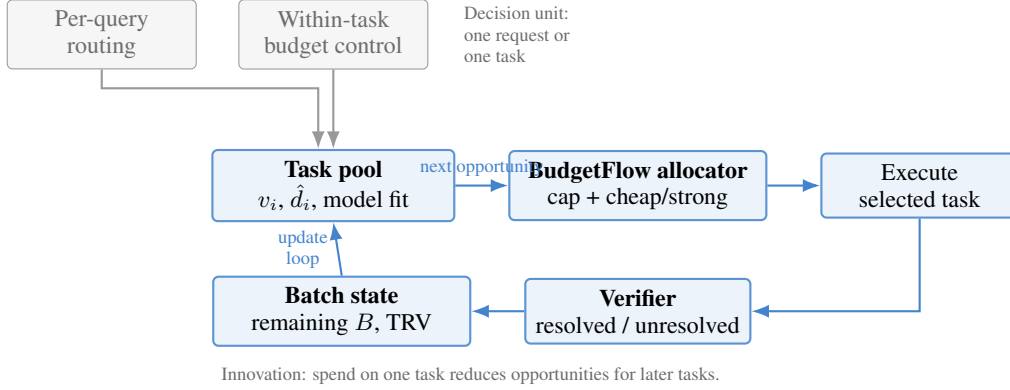


Figure 1. BudgetFlow overview. Light boxes mark adjacent decision units. The main loop allocates across a task pool under one shared hard budget: choose the next task opportunity from frozen Task Value, estimated token cost, and model fit; execute; settle with a Verifier; update remaining budget and verified TRV.

Cost-aware agent runtimes. OpenSquilla is a token-efficient agent runtime with model routing, diagnostics, replay, and cost-per-success framing (OpenSquilla contributors, 2026). It strengthens the requirement that agent cost and execution behavior be measurable. Closely related harness-evaluation work such as Claw-SWE-Bench further stresses fair harness comparison and cost reporting (Zheng et al., 2026), but it evaluates adapters rather than cross-task budget governance. BudgetFlow uses a different decision unit: many tasks share one hard budget, and scarce strong-model opportunities should flow toward tasks that create the most verified value under frozen Task Value.

3. Problem Formulation

Let a task batch contain tasks $\mathcal{T} = \{1, \dots, n\}$ and let B denote the Shared Budget. Each task i has a pre-registered value $v_i > 0$, an estimated token cost $d_i > 0$, and model-fit evidence $m_{i,k}$ for each model tier k . A policy π chooses a sequence of actions that spend budget on model calls and task attempts. Let $c_i(\pi)$ be the spend on task i , and let

$$\sum_{i=1}^n c_i(\pi) \leq B$$

be the shared hard-budget constraint.

The budget is shared at the batch level. A policy can spend more on one task only by leaving less budget for later tasks in the same batch. This coupling is what separates the problem from independent per-task routing: a locally reasonable strong-model call can be globally poor if it consumes budget needed by a higher-value task.

Each task has a Verifier V_i that maps the final task evidence to a binary outcome:

$$r_i(\pi) = V_i(\pi, i) \in \{0, 1\}.$$

The primary value objective is Total Resolved Value:

$$\text{TRV}(\pi) = \sum_{i=1}^n v_i r_i(\pi).$$

We report three headline metrics:

$$\text{ResolvedRate}(\pi) = \frac{1}{n} \sum_{i=1}^n r_i(\pi),$$

$$\text{TRVperDollar}(\pi) = \frac{\text{TRV}(\pi)}{\sum_i c_i(\pi)}.$$

3.1. Core Terms and Metric Reading

Table 1 gives the compact reading guide used throughout the paper. The metrics are intentionally simple: success is verifier-settled, value is pre-registered, and efficiency is value divided by spend. This makes the frontier interpretable through standard verified outcomes and a pre-registered value ledger.

Term	Reading
Batch-Level Task Budget Allocation	A task batch shares one hard budget; a 30-task run spends at most \$9.95 total.
Task Value	Pre-registered value gained when the task passes the Verifier.
Estimated Token Cost	Pre-run token-cost estimate used for cap planning.
Model Fit	Evidence about which model tier is likely to help this task.
Verifier	Domain outcome rule; SWE-bench uses a test-based harness.
Shared Budget	Hard batch-level resource constraint shared by all tasks.
Resolved Rate	Verifier-passed tasks divided by all tasks; 16/30 means 53.3%.
Total Resolved Value (TRV)	Sum of Task Value over verifier-passed tasks.
TRV per Dollar	TRV divided by total spend; TRV 18.0 at \$9.95 gives 1.81 TRV/\$.
Strong Coverage	Frontier mode where broad strong-model use is cost-effective under the shared budget.
Model Fit	Frontier mode where accurate task-to-model matching can form a strong frontier point.
Value-Aware Budget Flow	Central BudgetFlow frontier mode where task value, estimated token cost, model fit, and shared budget jointly govern allocation.

Table 1. Core terms and metrics. The examples show how to read the reported numbers.

The Verifier is an outcome settlement rule. The policy uses Task Value, Estimated Token Cost, Model Fit, and Shared Budget state for allocation; the Verifier settles the final outcome after execution. In our SWE-bench instantiation, V_i is the test-based harness. This gives a strict external success signal while preserving the task-batch allocation structure.

The interfaces have different portability requirements. Task Value must be fixed before execution. Estimated Token Cost is a pre-run planning estimate; final spend is measured after execution. Model Fit can come from catalog priors, historical outcomes, or calibration runs. The Verifier must be a trusted outcome rule for the domain. The Shared Budget is set at the batch level. Their concrete calibration changes by domain, but the allocation problem consumes the same interface.

This interface also gives a domain-neutral governance rule. The policy allocates budget according to declared task value, estimated token cost, model-fit evidence, and the shared budget. In an enterprise batch, a coding task, a support task, and a document task can therefore compete under the same budget contract once each supplies the required signals and verifier.

4. BudgetFlow

BudgetFlow maintains a Shared Budget ledger across the task batch. For each task, it estimates whether the next model opportunity should use the cheap model, the strong model, or defer additional spend. The ledger is hard: Shared Budget gates each planned action. The goal is to turn the general interface in Section 3 into a concrete value-aware allocation policy.

The policy uses three allocation inputs:

- v_i , the Task Value;
- d_i , the Estimated Token Cost;
- $m_{i,k}$, the Model Fit for tier k .

These inputs enter distinct decisions. Task Value prioritizes tasks that create more value if verified. Estimated Token Cost becomes cap planning: the policy estimates how much token cost a task may need before it can produce a useful artifact. Model Fit estimates whether the strong model is likely to improve the outcome relative to the cheap model. Shared Budget controls affordability and stop/defer decisions.

Let h denote budget pressure, with higher h indicating tighter Shared Budget state. Let k_c be the cheap model and k_s be the strong model. A simplified task-start score for giving task i a strong-model opportunity is

$$S_i = \frac{v_i \cdot \max(m_{i,k_s} - m_{i,k_c}, 0)}{\max(d_i, \epsilon)} \cdot q(h),$$

where d_i is the task’s Estimated Token Cost and $q(h)$ reduces strong-model access as Shared Budget tightens. The implementation uses this score with affordability checks, learned task-router priors, and hard-budget gates. In the SWE-bench implementation, Model Fit combines catalog priors, batch-level tier-fit evidence from completed outcomes, and learned-router priors when available. The formula captures the core allocation principle: strong-model budget is most valuable when a task has high value, sufficient expected model-fit gain, manageable Estimated Token Cost, and enough Shared Budget.

This makes BudgetFlow a governance layer over task categories. The score can favor a coding issue, a spreadsheet repair, or a support response for the same reason: the task has high declared value, a feasible token-cost estimate, useful model-fit upside, and enough shared budget to make the model opportunity affordable.

BudgetFlow separates allocation from outcome settlement. It uses Task Value, Estimated Token Cost, Model Fit, and Shared Budget state to choose model opportunities. The

Interface	SWE-bench instantiation	Other task-batch instantiations
Task Value	pre-registered criticality value	business priority, user tier, task owner value
Estimated Token Cost	metadata-only estimates or prior-batch observations	expected context size, tool steps, artifact size, or prior workload traces
Model Fit	catalog priors, prior outcomes, or router priors	domain-specific success prior by model tier
Verifier	test-based harness	human review, rule check, delayed business outcome
Shared Budget	batch-level cap in dollars	team budget, project budget, processing window budget

Table 2. Domain interfaces for Batch-Level Task Budget Allocation. The paper evaluates the SWE-bench instantiation.

Algorithm 1 BudgetFlow batch policy sketch

```

1: Initialize Shared Budget state  $R \leftarrow B$ .
2: for task  $i$  in the batch do
3:   Read  $(v_i, d_i, m_{i,k_e}, m_{i,k_s})$  and current  $R$ .
4:   Set an initial task cap from Task Value, Estimated Token Cost, and  $R$ .
5:   Choose cheap or strong model from Model Fit, value upside, and affordability.
6:   Execute the task while updating spend and Shared Budget.
7:   Continue, stop, or defer using value upside and budget pressure.
8:   Settle the final artifact with  $V_i$  and record  $r_i$ .
9: end for
10: Report Resolved Rate, TRV, TRV per Dollar, and total spend.
    
```

Verifier evaluates the resulting patch or artifact after the task attempt. This design keeps the value objective auditable: the policy can explain why a task received budget, and the verifier independently determines whether that spend produced resolved value.

Allocation decisions. At task start, BudgetFlow assigns a route and an initial cap from value, Estimated Token Cost, Model Fit, and Shared Budget. During execution, it can assign a strong-model opportunity when the strong model has expected upside and Shared Budget allows it. It can stop or defer when additional spend has low expected value or would threaten the batch budget. These decisions are task-level in the current paper; finer stage or segment routing is left for future work.

Algorithm 1 is the implementation-facing sketch of the framework. The policy reads the three allocation interfaces before and during execution, while the Verifier is applied after the task attempt. This keeps routing, cap planning, and strong-model opportunity decisions auditable under the same value ledger used for evaluation.

5. Experimental Setup

Task set. We evaluate on fixed 30-task SWE-bench-style bug-fixing batches. Each task requires an agent to inspect a repository, edit code, and produce a patch. Outcomes are scored by a local test-based harness, and every run exports official-format predictions for independent re-evaluation. Task values are pre-registered before execution.

Policies. Table 3 compares the five policies by the inputs they use. All policies run under the same shared hard budget and the same verifier. The key contrast is whether Task Value enters allocation of scarce model opportunities.

The 4-policy run uses the four policies available in that experimental line. All mainline policies use the same task order, budget protocol, model-tier catalog, and scoring path inside a run.

Metrics. The main tables report Resolved Rate, TRV, and TRV per Dollar, as defined in Table 1. Resolved Rate is written as both count and percentage. Spend is included as budget context because TRV per Dollar depends on it.

Sensitivity analyses. The main paid runs show observed frontier points. The sensitivity analyses do not add new paid runs or cross-run frontier points; they replay one completed batch under alternate accounting assumptions. Task Value sensitivity re-scores fixed outcomes under alternate value profiles. Budget sensitivity replays the observed task order under sampled shared-budget caps. KV Cache Cost-Discount sensitivity re-costs repeated input tokens under explicit cache-discount assumptions. Together, these diagnostics separate outcome quality from the accounting assumptions that determine cost-value efficiency.

6. Results

6.1. Main Paid Evidence

Table 5 summarizes four paid task-batch runs through the BudgetFlow framework, with configuration details in Table 4. Red entries mark BudgetFlow frontier signals; blue

BudgetFlow: Value-Aware Budget Governance for Agent Tasks

Policy	Shared budget	Task Value	Budget planning input	Model choice	Same verifier
cheap-model-only	yes		shared cap	fixed cheap model	yes
strong-model-only	yes		shared cap	fixed strong model	yes
learned task-router	yes		shared cap	value-blind learned choice	yes
budget-only	yes		budget pressure	budget-pressure choice	yes
BudgetFlow	yes	yes	value-aware task budgets	value-aware choice	yes

Table 3. Policy inputs and shared outcome contract. Budget planning input describes what each policy uses beyond the shared hard cap.

Batch	Policies	Planning input	Catalog	Value ledger	Hard cap
4-policy BudgetFlow Pareto case	4	metadata-only estimates	2026-06-22 T2/T3x5	10 high-value; max 2.5	\$10.44 stage-prefix pressure
5-policy Strong Coverage larger-lead case	5	metadata-only estimates	2026-06-22 T2/T3 catalog	10 high-value; max 2.5	\$9.95 strongest spend divided by 0.95
5-policy Strong Coverage close-lead case	5	prior-batch observations	2026-06-30 T2/T3 catalog	6 high-value; max 1.5	\$9.95 strongest spend divided by 0.95
5-policy BudgetFlow leads-all case	5	prior-batch observations	2026-06-30 T2/T3 catalog	6 high-value; max 1.5	\$9.95 strongest spend divided by 0.95

Table 4. Run configuration summary. Planning input identifies whether Estimated Token Cost and Model Fit use metadata-only estimates or prior-batch observations. The batches are interpreted separately because catalog revision and value ledger differ.

entries mark the strongest control when a control defines the observed frontier. The first column states the intended reading of each batch so the table is not read as a cross-run leaderboard.

The first value-aware result is the 4-policy BudgetFlow Pareto case. BudgetFlow reaches **TRV 18.50 at \$7.98**, while strong-model-only reaches TRV 18.00 at \$9.46. This is a strict improvement in the (spend, TRV) plane—higher resolved value at about 16% lower spend—while strong-model-only resolves one additional task (15/30 vs. 14/30). The value gain is concentrated where the value ledger says it should be: on high-value tasks (Task Value ≥ 1.5), BudgetFlow resolves 9.50 value against 7.00 for strong-model-only.

The second value-aware result is the 5-policy BudgetFlow leads-all case. BudgetFlow resolves 16/30 tasks, reaches TRV 18.00, and achieves 1.81 TRV per Dollar. The next-best learned task-router resolves 15/30 tasks, reaches TRV 17.00, and achieves 1.71 TRV per Dollar. The two 5-policy Strong Coverage cases show the other main pattern: in both, strong-model-only forms the observed frontier, but the margin is large in one batch and close in the other.

The four main batches show two frontier patterns. First, Value-Aware Budget Flow can protect high-value tasks and **improve TRV and TRV per Dollar** even when a strong-model policy resolves one additional task. Second, Strong Coverage explains cases where **a strong model is already a strong frontier point** because broad strong-model coverage is cost-effective under the cap. The two Strong Coverage rows are the same condition at two margins, not two different mechanisms. The 5-policy BudgetFlow leads-all case shows the desired BudgetFlow frontier movement: BudgetFlow **leads on standard resolution, value, and value per dollar** under the same cap.

These outcomes should be read together. The 4-policy BudgetFlow Pareto case and the 5-policy BudgetFlow leads-all case are the two strongest value-aware examples: the former shows higher value efficiency despite one fewer resolved task, and the latter shows BudgetFlow leading all three primary metrics. In one Strong Coverage case, strong-model-only has a larger lead; in the other, BudgetFlow is close to that frontier, trailing by 0.50 TRV and 0.06 TRV per Dollar. The Model Fit case appears in serving-cost sensitivity, where learned routing becomes the best TRV-per-Dollar point under extreme KV-cache discounts. Table 6 maps the resulting frontier modes to representative evidence.

6.2. Frontier Mode Analysis

Table 6 maps each frontier mode to representative evidence. This analysis is the interpretation layer for the same Resolved Rate, TRV, and TRV per Dollar evidence in Table 5. A shared-budget batch can be frontier-shaped by different signals: value-aware budget flow, broad strong-model coverage, or accurate model-fit routing. The BudgetFlow contribution is to make Value-Aware Budget Flow concrete through a framework and interface for value-governed allocation.

6.3. Full 5-Policy Value-Aware Budget Flow Case

Table 7 gives the full 5-policy Value-Aware Budget Flow case. **BudgetFlow leads all three primary metrics**. This is the clearest point-wise evidence for the current BudgetFlow policy instantiation; Section 7 reports sensitivity curves for alternate frontier modes.

BudgetFlow: Value-Aware Budget Governance for Agent Tasks

Batch reading	Policy	Resolved Rate	Spend	TRV	TRV per Dollar
BudgetFlow Pareto: higher TRV at lower spend	BudgetFlow	14/30 (46.7%)	\$7.98	18.50	2.32
	strong-model-only	15/30 (50.0%)	\$9.46	18.00	1.90
Strong Coverage: larger strong-model lead	strong-model-only	17/30 (56.7%)	\$9.95	20.00	2.01
	BudgetFlow	15/30 (50.0%)	\$9.95	17.50	1.76
Strong Coverage: close strong-model lead	strong-model-only	18/30 (60.0%)	\$9.88	19.00	1.92
	BudgetFlow	17/30 (56.7%)	\$9.95	18.50	1.86
BudgetFlow leads all: all primary metrics	BudgetFlow	16/30 (53.3%)	\$9.95	18.00	1.81
	learned task-router	15/30 (50.0%)	\$9.95	17.00	1.71
	strong-model-only	15/30 (50.0%)	\$9.95	16.50	1.66

Table 5. Main paid evidence. Each group reports one 30-task batch. The two BudgetFlow-leading groups show the value-aware cases: one improves the spend–TRV frontier, and one leads all primary metrics. The two Strong Coverage groups should be read together: in both, broad strong-model use is cost-effective under the cap; larger and close describe the observed lead. Red bold entries mark BudgetFlow frontier signals; blue bold entries mark the strongest control in Strong Coverage runs. The 4-policy BudgetFlow row uses total observed spend, including non-scoreable abort overhead.

Frontier mode	Condition	Representative evidence	Reading
Value-Aware Budget Flow	Task values differ, estimated token costs differ, and model gains are selective.	Main paid evidence: BudgetFlow reaches 18.50 TRV and 2.32 TRV/\$ in the BudgetFlow Pareto case, and 16/30, 18.00 TRV, 1.81 TRV/\$ in the BudgetFlow leads-all case.	Central BudgetFlow mode.
Strong Coverage	The strong model covers the batch broadly under the cap.	Main paid evidence: strong-model-only reaches 17/30, 20.00 TRV, 2.01 TRV/\$ in the larger-lead case, and 18/30, 19.00 TRV, 1.92 TRV/\$ in the close-lead case.	Strong-model frontier point.
Model Fit	The learned routing signal places model tiers accurately.	Serving-cost sensitivity: at KV98 and KV99, the learned task-router reaches 14.28 and 15.41 TRV/\$, ahead of BudgetFlow on TRV per Dollar while BudgetFlow keeps higher TRV.	Model-fit frontier point under extreme cache discount.

Table 6. Frontier modes and representative evidence. Main paid evidence establishes the Value-Aware Budget Flow and Strong Coverage cases; serving-cost sensitivity supplies the Model Fit frontier point.

Policy	Resolved Rate	TRV	TRV/\$
BudgetFlow	16/30 (53.3%)	18.00	1.81
learned task-router	15/30 (50.0%)	17.00	1.71
strong-model-only	15/30 (50.0%)	16.50	1.66
budget-only	12/30 (40.0%)	13.00	1.31
cheap-model-only	11/30 (36.7%)	12.00	1.21

Table 7. 5-policy Value-Aware Budget Flow case over 30 tasks.

6.4. Task-Level Model Advantage

The 5-policy Value-Aware Budget Flow case contains real model-tier heterogeneity. Among tasks where both cheap and strong model rows consumed model budget, there are **six cheap-model-favorable passes, five strong-model-favorable passes, and seven comparable failures under both**. This is the mechanism-level reason a batch-budget

framework is useful: the task batch contains cheap-model opportunities, strong-model opportunities, and ceiling tasks in the same Shared Budget.

This task-level heterogeneity motivates selective strong-model use. When strong-model upside is concentrated on a subset of tasks, the allocation policy should preserve strong-model opportunities for those tasks while spending less on tasks where the cheap model is sufficient or both tiers are unlikely to pass.

7. Sensitivity Analysis

The sensitivity section has a different role from the main paid evidence. The main evidence reports observed paid batches; sensitivity analysis reuses one completed batch and shows how the same outcomes move under alternate value,

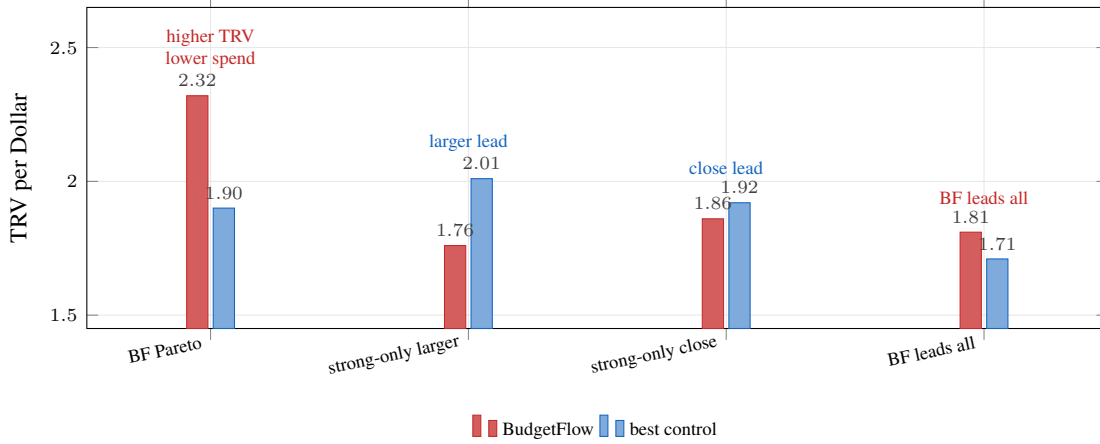


Figure 2. Main evidence compressed to value efficiency. Each pair compares BudgetFlow with the best available control from the same paid batch on TRV per Dollar. The first and fourth pairs are the value-aware BudgetFlow cases; the middle pairs are Strong Coverage cases, with one larger strong-model lead and one close lead.

Value profile	BF TRV	Control TRV	Margin
equal	16.00	15.00	+1.00
criticality	18.00	17.00	+1.00
compressed	17.00	16.00	+1.00
expanded	20.00	19.00	+1.00

Table 8. Task Value sensitivity on the 5-policy Value-Aware Budget Flow case.

budget, and serving-cost assumptions. We therefore treat sensitivity as a frontier diagnostic: each table or curve explains how the frontier mode changes when the accounting environment changes.

7.1. Task Value Sensitivity

Task Value sensitivity tests the Value-Aware Budget Flow mode under alternate ledgers. Table 8 re-scores the 5-policy Value-Aware Budget Flow case under alternate Task Value profiles. BudgetFlow remains ahead of the best control under equal value, criticality value, compressed criticality, and expanded criticality. A value permutation diagnostic over 64 permutations gives BudgetFlow a positive margin in all samples, with median margin +1.00 TRV. The conclusion is that this frontier movement is visible across several value ledgers, including the plain resolved-count setting where all resolved tasks have equal value.

7.2. Budget Sensitivity

Budget sensitivity is the closest analogue to a frontier plot. Figure 3 fixes observed outcomes and replays budget accounting at sampled caps. This is not a new lower-cap planning run: the replay keeps the completed full-cap task order, outcomes, and accounting rules, then asks what would

remain under tighter caps. It is also not a cross-run plot; all points come from the same completed 5-policy Value-Aware Budget Flow case. The strong-model-only policy is strongest at some low and mid replay caps, while BudgetFlow reaches the replay frontier at the largest sampled cap. The important reading is the frontier shape: budget pressure can move a batch between Strong Coverage, Model Fit, and Value-Aware Budget Flow modes, and BudgetFlow provides the framework and interface that make this movement reportable.

7.3. KV Cache Cost-Discount Sensitivity

KV Cache Cost-Discount sensitivity tests a serving-cost question. Modern serving substrates can make repeated input context cheaper; this changes effective spend while leaving verified outcomes unchanged. Table 9 re-costs repeated input tokens under KV-cache discounts. BudgetFlow leads or stays close through KV90. At KV98 and KV99, the learned task-router becomes the Model Fit frontier point by TRV per Dollar, while BudgetFlow keeps a higher TRV than the learned task-router in the fixed-outcome table.

This pattern gives a useful serving-cost reading. BudgetFlow gains part of its efficiency by allocating scarce budget across tasks. When cache discounts become extreme, the cost penalty of repeated or stronger model use shrinks, and accurate task-to-model matching can become more competitive on TRV per Dollar. The result makes the serving boundary explicit: BudgetFlow is most useful under meaningful budget pressure, and serving optimizations can move the batch toward Strong Coverage or Model Fit frontier modes.

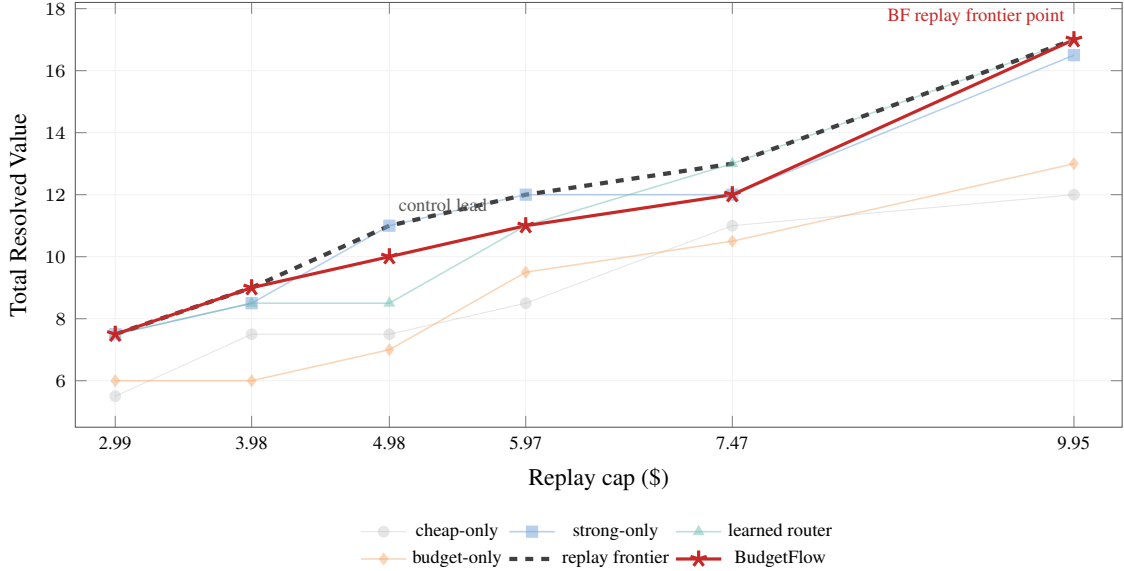


Figure 3. Budget sensitivity diagnostic on the 5-policy Value-Aware Budget Flow case. Outcomes are fixed; budget accounting is replayed at sampled caps without re-planning task order or routing. The dashed line is the replay frontier for this completed batch; BudgetFlow is emphasized in red, while controls are shown as lighter diagnostic curves.

KV profile	BF	Learned router	BF margin
KV0	1.81	1.71	+0.10
KV50	3.26	3.14	+0.11
KV90	9.06	8.98	+0.09
KV98	14.08	14.28	-0.20
KV99	15.13	15.41	-0.29

Table 9. KV Cache Cost-Discount sensitivity on the 5-policy Value-Aware Budget Flow case. Values are TRV per Dollar; the learned task-router is the best control in this fixed-outcome re-costing.

8. Discussion and Future Work

Formulation-level generalization. The SWE-bench experiments instantiate the formulation in a verified coding domain. The formulation applies to task domains where tasks expose Task Value, Estimated Token Cost, Model Fit, and a Verifier. Coding tasks use tests as the Verifier. Support tasks can use policy checks and service outcomes. Spreadsheet tasks can use cell-level checks or rule engines. Research and document-production tasks can use human review or domain checks. The calibration of each interface is domain-specific; the allocation objective and budget ledger are shared. This is the practical generalization claim: the task source changes, while the budget-governance contract remains the same.

Frontier-aware reporting. The same BudgetFlow framework explains multiple frontier positions across the paid runs. Strong Coverage captures the common intuition that a broadly effective strong model can be worth its cost. Model

Fit captures the learned-routing intuition that the best tier depends on the task. Value-Aware Budget Flow captures the BudgetFlow contribution: when tasks have unequal value and compete for one shared budget, the policy must decide where scarce strong-model opportunities create the most verified value. Reporting frontier modes turns these cases into an operating guide for shared-budget agent batches.

Future work. Fine-grained task execution control. BudgetFlow can move from task-level allocation to finer-grained execution control inside a task. The current paper assigns model opportunities at the task level. A richer policy could decide which task stage or segment deserves the strong model, how much repair runway to preserve, and when stop/defer decisions should be made inside a long attempt. This direction should be evaluated against task-level controls because finer granularity can improve allocation only when the stage signal is reliable enough to justify the added switching cost.

Continual policy learning with auditability. Completed batches produce cost traces, routing outcomes, verifier results, and frontier positions. **A future policy can use those records to update future caps, route choices, and stop/continue decisions.** The important systems constraint is that each learned update should remain inspectable: the next allocation should be explainable through the same value ledger, Estimated Token Cost estimate, model-fit evidence, Shared Budget state, and verifier-settled outcome history.

Serving-aware and multi-tenant BudgetFlow. Serving systems such as vLLM, SGLang, and Dynamo optimize

execution substrates: batching, cache locality, scheduling, and inference throughput. BudgetFlow operates at the task-budget layer above that substrate. A serving-aware BudgetFlow could pass task value, Shared Budget state, Estimated Token Cost, and outcome history into scheduling hints such as priority, continuation, model opportunity, and cache-sticky execution. The same interface can then extend from one budget owner to multiple teams, users, or services that share an agent execution substrate with distinct budgets and priorities. **This turns Batch-Level Task Budget Allocation into a candidate control plane for multi-tenant agent workloads.**

9. Conclusion

This paper formulates Batch-Level Task Budget Allocation for multi-step agent tasks under a shared hard budget. The core question is which task should receive the next scarce model opportunity when tasks have different value, different estimated token cost, and different model-tier fit. BudgetFlow answers this question with a simple auditable interface: Task Value, Estimated Token Cost, Model Fit, Shared Budget state, and a Verifier-settled outcome.

The practical role of TRV is value accounting. It is a frozen value score that asks whether the same spend resolves more pre-declared task value. This makes mixed frontier positions meaningful: the paper reports which frontier mode shapes the cost-value frontier under a Shared Budget.

The SWE-bench experiments instantiate this interface in a verified coding domain. In the 5-policy Value-Aware Budget Flow case, BudgetFlow **improves Resolved Rate, TRV, and TRV per Dollar**. In the 4-policy Value-Aware Budget Flow case, BudgetFlow resolves one fewer task than strong-model-only while producing higher TRV and higher TRV per Dollar. Across the broader paid evidence, the frontier is explained by three frontier modes. Strong Coverage describes batches where broad strong-model use is cost-effective under the cap. Model Fit describes batches where learned routing accurately places model tiers. Value-Aware Budget Flow describes the central BudgetFlow setting: task values are uneven, model-tier advantage is selective, and the Shared Budget forces cross-task tradeoffs.

The broader value of BudgetFlow is the framework and interface. Coding tasks provide a strict verifier and measurable spend, making SWE-bench a strong testbed for the allocation contract. The transferable object is the interface: tasks expose Task Value, Estimated Token Cost, Model Fit, Shared Budget state, and verifier-settled outcomes. By **separating allocation signals from outcome settlement**, BudgetFlow offers a portable way to reason about verified value under fixed resource constraints, from SWE-bench-style coding to future serving-aware and multi-tenant agent

systems.

References

- Dekoninck, J., Baader, M., and Vechev, M. A unified approach to routing and cascading for LLMs. In *International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2410.10347>.
- Guo, D., Wu, J., and Yiu, S. M. RouteNLP: Closed-loop LLM routing with conformal cascading and distillation co-optimization. In *ACL Industry Track*, 2026. URL <https://arxiv.org/abs/2604.23577>.
- Kotte, V. UCCI: Calibrated uncertainty for cost-optimal LLM cascade routing. *arXiv preprint arXiv:2605.18796*, 2026. URL <https://arxiv.org/abs/2605.18796>.
- Lee, S., Kim, D., Koh, H., Yang, N., and Jung, K. Confidence-guided stepwise model routing for cost-efficient reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://arxiv.org/abs/2511.06190>.
- Liu, H., Tian, C., An, N., Wang, Z., Lu, P., Yu, C., and Qi, Q. Budget-constrained agentic large language models: Intention-based planning for costly tool use. *arXiv preprint arXiv:2602.11541*, 2026. URL <https://arxiv.org/abs/2602.11541>.
- Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2406.18665>.
- OpenSquilla contributors. OpenSquilla: Token-efficient ai agent. <https://github.com/opensquilla/opensquilla>, 2026. Accessed 2026-07-06.
- Poon, M., Dai, X., Liu, X., Kong, F., Lui, J. C. S., and Zuo, J. Online multi-LLM selection via contextual bandits under unstructured context evolution. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://arxiv.org/abs/2506.17670>.
- Si, W., Jang, S., Lee, I., and Bastani, O. Conformal constrained policy optimization for cost-effective LLM agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- Su, J., Lin, F., Feng, Z., Zheng, H., Wang, T., Xiao, Z., Zhao, X., Liu, Z., Cheng, L., and Wang, H. CP-Router: An uncertainty-aware router between LLM and LRM. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

- Wang, C., Li, H., Zhang, Y., Chen, L., Chen, J., Jian, P., Zhang, Q., and Hu, S. ICL-Router: In-context learned model representations for LLM routing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- Yan, C., Zhang, W., Ning, Z., Xu, F., Tao, Z., Zhang, L., Yin, B., and Zhang, Y. Breaking model lock-in: Cost-efficient zero-shot LLM routing via a universal latent space. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://arxiv.org/abs/2601.06220>.
- Yang, L., Luo, J., Liu, X., Lou, Y., and Chen, Z. BAMA-S: Structuring budget-aware multi-agent systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40226>.
- Zhang, Y.-K., Zhan, D.-C., and Ye, H.-J. Capability instruction tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.
- Zheng, M., Han, K., Li, B., Xu, H., Tian, Y., He, W., Zhou, H., Guo, J., Hu, H., Ma, L., Xu, C., Dai, G., Xia, L., Wei, Y., Wang, Y., and Wang, Y. Claw-SWE-Bench: A benchmark for evaluating openclaw-style agent harnesses on coding tasks. *arXiv preprint arXiv:2606.12344*, 2026. URL <https://arxiv.org/abs/2606.12344>.